Documentation technique :

"Network Random Compression"

Aina Arimanana RAHARISON

20 Juillet 2026

Contents

1 Introduction **4**

2 Guide d'installation **5**
2.1 Environnement logiciel 5
2.2 Choix et téléchargement du jeu de données 6

3 Guide d'utilisation **7**
3.1 Exemples d'utilisation 7
3.1.1 Entraînement de référence 7
3.1.2 Balayage(ou sweep) complet 7
3.1.3 Utilisation via un fichier de configuration 8
3.1.4 Test de compression sur plusieurs modèles 9
3.2 Lecture des résultats 10
3.2.1 Gain de compression 10
3.2.2 Écart d'accuracy 10
3.2.3 Coût de calcul (MAC) 10
3.3 Ajout d'un nouveau modèle 11

4 Vue d'ensemble du flot de conception **13**
4.1 Validation par des balayages des paramètres 14

5 Bloc 1 — Entraînement du modèle initial **14**
5.1 Objectif 14
5.2 Fichiers concernés 15
5.3 Entrées et sorties 16
5.4 Paramètres de configuration 17

6 Bloc 2 — Analyse PCA des filtres **17**
6.1 Objectif 17
6.2 Fichier concerné 17
6.3 Centrage et réduction de dimension 17
6.4 Entrées, sorties et métriques 19
6.5 Paramètre principal 19

7 Bloc 3 — Pseudo-randomisation LFSR et distance de Grassmann **20**
7.1 Objectif 20
7.2 Fichiers concernés 20

7.3 Répartition des vecteurs 20
7.4 Sélection des vecteurs pseudo-aléatoires 20
7.5 Distance de Grassmann 23
7.6 Entrées, sorties et métriques 23
7.7 Paramètres de configuration 24

8 Bloc 4 — Reconstruction de couches de convolutions (ou NRCConvBlock) 24
8.1 Objectif 24
8.2 Fichiers concernés 24
8.3 Formulation 24
8.4 Composition du bloc NRCConvBlock 25
8.5 Entrées, sorties et métriques 25
8.6 Paramètres de configuration 26

9 Bloc 5 — Fine-tuning partiel 26
9.1 Objectif 26
9.2 Fichiers concernés 26
9.3 Équité de comparaison 26
9.4 Entrées, sorties et métriques 27
9.5 Paramètres de configuration 28

10 Bloc 6 — Sérialisation, restauration et bilan 28
10.1 Objectif 28
10.2 Fichiers concernés 29
10.3 Fichiers produits 29
10.4 Code couleur du fichier Excel 30
10.5 Entrées, sorties et métriques 30
10.6 Paramètres de configuration 31
10.7 Bilan expérimental 31

11 Conclusion 32

Liste des codes

1 Installation des dépendances du pipeline NRC. 6
2 Registre des jeux de données (`run.py`). 6
3 Téléchargement automatique du jeu de données choisi (`run.py`). 6
4 Entraînement du modèle de référence seul. 7

5 Balayage complet de la grille $eTh \times p$ avec fine-tuning et sauvegarde. 7
6 Lancement du pipeline à partir d'un fichier de configuration. . 8
7 Contenu de `config_exemple.txt`. 9
8 Test de compression sur plusieurs modèles (`test_all_models.py`). 9
9 Squelette minimal d'un nouveau modèle CIFAR-10. 12
10 Enregistrement du modèle dans `MODEL_REGISTRY`. 13

Index

CIFAR-10	Canadian Institute For Advanced Research (10 classes)	Jeu de données de référence (60 000 images 32 × 32 RGB, 10 classes) utilisé pour l'entraînement et l'évaluation
CNN	Convolutional Neural Network	Réseau de neurones convolutif
FC	Fully Connected	Couche entièrement connectée (`nn.Linear`), non compressé par NRC, gelable via `–freeze-fc`
LFSR	Linear Feedback Shift Register	Registre à décalage utilisé pour la génération pseudo-aléatoire de filtres (Bloc 3)
MAC	Multiply-ACcumulate	Opération multiplication-accumulation ; unité de mesure du coût de calcul d'une convolution
NRC	Network Random Compression	Méthode de compression décrite dans ce document, combinant PCA et pseudo-randomisation LFSR
PCA	Principal Component Analysis	Méthode de réduction de dimension par décomposition en composantes principales
QR	Décomposition matricielle QR	Factorisation orthogonale (`numpy.linalg.qr`) utilisée pour orthonormaliser les bases lors du calcul de la distance de Grassmann (Bloc3)
SVD	Singular Value Decomposition	Décomposition en Valeurs singulières utilisée pour extraire les angles principaux de la distance de Grassmann (Bloc3)

1 Introduction

Cette documentation présente le flot de conception et de mise en œuvre de la méthode **NRC (Network Random Compression)**. Cette méthode vise à compresser les réseaux de neurones convolutifs (CNN) en exploitant une ensemble ou une base hybride des filtres de convolutions composée de vecteurs ou des filtres de convolutions issus d'une analyse en composantes principales (PCA) et de vecteurs pseudo-aléatoires générés à partir d'un registre à décalage à rétroaction linéaire (*Linear Feedback Shift Register*, LFSR).

L'objectif principal est de réduire le nombre de paramètres nécessaires à la représentation des couches convolutives tout en préservant au maximum les performances du réseau.

La méthode NRC repose sur une factorisation des couches `Conv2d` dans une base hybride notée B_{NRC}. Cette base est composée de deux types de vecteurs :

- les vecteurs propres conservés issus de PCA, notés V_s, sont stockés explicitement ;
- puis certains de ces vecteurs sont remplacés par les vecteurs pseudo-aléatoires, notés V_r, qui peuvent être régénérés à partir d'une graine LFSR de 4 octets.

Deux hyperparamètres principaux permettent de contrôler le compromis entre la précision du modèle et le gain mémoire :

- *eTh* : seuil d'énergie utilisé lors de l'analyse PCA pour déterminer le nombre de dimensions des filtres de convolution **s** conservées ;
- *p* : proportion des remplacement des vecteurs conservées issus de PCA qui seront remplacées par des vecteurs pseudo-aléatoires.

Le code de référence associé à cette documentation est organisé autour des fichiers `run.py`, `pipeline.py`, `utils.py` ainsi que des répertoires `nrc/` et `models/`.

2 Guide d'installation

2.1 Environnement logiciel

L'ensemble du pipeline est implémenté en **Python 3.12** (environnement testé sur ***python 3.12.10***). Le tableau suivant recense les bibliothèques et frameworks effectivement utilisés, avec leurs versions figées dans `requirements.txt`.

Bibliothèque	Version	Rôle
`torch`	2.10.0+cu126	Framework principal utilisé pour définir les modèles `nn.Module`, les couches `Conv2d`/`Linear`, l'optimiseur Adam et l'exécution sur CPU/CUDA.
`torchvision`	0.25.0+cu126	Bibliothèque complémentaire de PyTorch utilisée notamment pour le chargement du jeu de données CIFAR-10 et les transformations d'images.
`numpy`	2.4.3	Calcul matriciel (centrage, produits matriciels, QR, SVD), génération des vecteurs LFSR, sérialisation
`scikit-learn`	1.8.0	Analyse en composantes principales (`sklearn.decomposition.PCA`) pour extraire la base B_T et les valeurs propres
`pandas`	3.0.1	Construction des tableaux de résultats et export vers Excel (`pandas.ExcelWriter`)
`openpyxl`	3.1.5	Lecture et écriture des fichiers Excel utilisées par pandas (`engine="openpyxl"`) et mise en forme des cellules dans Excel

Table 2: Bibliothèques et frameworks utilisés dans le pipeline NRC

`torch` et `torchvision` utilisent une variante CUDA (`cu126`) que l'index PyPI par défaut ne résout pas ; ils doivent donc être installés manuellement depuis l'index PyTorch dédié **avant** le reste des dépendances, qui s'installe normalement depuis `requirements.txt` :

```
# 1. torch / torchvision (la commande suivante doit être supposée
    n'être qu'une seule ligne)
```

```
pip install torch==2.10.0+cu126 torchvision==0.25.0+cu126
    --index-url https://download.pytorch.org/whl/cu126

# 2. reste des dependances (numpy, scikit-learn, pandas, openpyxl)
pip install -r requirements.txt
```

Listing 1: Installation des dépendances du pipeline NRC.

2.2 Choix et téléchargement du jeu de données

Le pipeline ne se limite pas à CIFAR-10 : l'option –dataset sélectionne le jeu de données parmi un registre DATASET_REGISTRY (run.py), construit sur le même principe que MODEL_REGISTRY (§3.3) :

```
DATASET_REGISTRY = {
    "cifar10":  {"cls": torchvision.datasets.CIFAR10,
    "num_classes": 10},
    "cifar100": {"cls": torchvision.datasets.CIFAR100,
    "num_classes": 100},
}
```

Listing 2: Registre des jeux de données (run.py).

Le jeu de données choisi est téléchargé automatiquement au premier lancement de run.py grâce à torchvision :

```
dataset_cls = DATASET_REGISTRY[args.dataset]["cls"]
trainset = dataset_cls(
    root="./data", train=True,  download=True, transform=transform)
testset  = dataset_cls(
    root="./data", train=False, download=True, transform=transform)
```

Listing 3: Téléchargement automatique du jeu de données choisi (run.py).

Le paramètre download=True déclenche le téléchargement de l'archive correspondante (cifar-10-python.tar.gz ou cifar-100-python.tar.gz, taille comparable, environ 170 Mo) dans ./data puis son extraction ; les lancements suivants réutilisent directement les fichiers déjà présents sans re-télécharger. Aucune action manuelle n'est donc requise pour tester la plateforme sur une machine vierge : il suffit d'une connexion internet lors de la toute première exécution.

Pourquoi CIFAR-100 et pas d'autres datasets ? CIFAR-100 a été retenu comme premier jeu de données additionnel car il partage *exactement* le même format que CIFAR-10 : images 32×32 en couleur (3 canaux), même API torchvision (root,

`train`, `download`, `transform`). Seul le nombre de classes change (100 au lieu de 10), ce qui ne modifie ni la taille d'entrée des modèles, ni le calcul du coût MAC (§3.2.3, dont la taille d'entrée factice reste figée à 32×32), ni la structure des couches convolutives compressées par NRC : seule la dernière couche `nn.Linear` de chaque modèle doit s'adapter, via le paramètre `num_classes` désormais propagé depuis `args.dataset` jusqu'à `MODEL_REGISTRY`. Ce choix minimise donc le risque de régression tout en permettant d'évaluer la robustesse de la compression NRC sur un problème de classification nettement plus fin (100 classes contre 10, soit dix fois moins d'exemples par classe à volume de données égal).
D'autres jeux de données du même type (images naturelles en couleur) existent — SVHN, STL-10 — mais n'ont pas été retenus dans un premier temps : SVHN partage la résolution 32×32 mais expose une API `torchvision` différente (`split="train"/"test"` au lieu de `train=True/False`), et STL-10 utilise une résolution 96×96 qui nécessiterait d'adapter les couches `Linear` des modèles (dimensions actuellement calculées en dur pour une entrée 32×32) ainsi que la taille d'entrée factice de `count_conv_macs`. Le registre `DATASET_REGISTRY` est conçu pour accueillir ces extensions ultérieurement sans modifier le reste du pipeline.

3 Guide d'utilisation

3.1 Exemples d'utilisation

3.1.1 Entraînement de référence

```
python run.py --model lenet --baseline-only --epochs-initial 4
```

Listing 4: Entraînement du modèle de référence seul.

3.1.2 Balayage(ou sweep) complet

(la commande suivante est supposée n'être qu'une seule ligne)

```
python run.py --model vgg16 --eth 0.25 0.50 0.75 0.90 --p 0.0 0.25
0.50 0.75 --retrain-after-pca --retrain-after-pseudo
   --epochs-initial 4 --epochs-retrain-pca 2
   --epochs-retrain-pseudo 2 --save-models
```

Listing 5: Balayage complet de la grille *eTh* $\times$ *p* avec fine-tuning et sauvegarde.

3.1.3 Utilisation via un fichier de configuration

Plutôt que d'énumérer toutes les options en ligne de commande, il est possible de les rassembler dans un fichier texte et de le passer à `run.py` via `–config` (ou directement en argument préfixé par `@`) :

```
python run.py --config config_exemple.txt
# --- ou, de manière équivalente ---
python run.py @config_exemple.txt
```

Listing 6: Lancement du pipeline à partir d'un fichier de configuration.

Le fichier accepte deux styles d'écriture, mélangeables au sein d'un même fichier (`–option valeur`, identique à la ligne de commande, ou `option = valeur`, les caractères `_`/`-` étant équivalents) ; tout ce qui suit un `#` est un commentaire, les lignes vides sont ignorées, et un argument donné explicitement en ligne de commande écrase toujours celui du fichier. Le fichier `config_exemple.txt` fourni avec le dépôt illustre l'ensemble des options du pipeline :

```
# -- Modèle et dataset ------------------------------------------------
model   = lenet   # lenet | vgg16 | vgg16_standard | resnet18 |
    resnet50 | vgg_mini
dataset = cifar10 # cifar10 | cifar100 (meme format 32x32 RGB,
    num_classes s'adapte)

# -- Grille de balayage (listes de valeurs séparées par des espaces)
eth = 0.25 0.50 0.75 0.90   # seuils d'énergie PCA
p   = 0.00 0.25 0.50        # proportions de filtres
    pseudo-randomisés

# -- Ré-entraînement --------------------------------------------------
# retrain_after_pca : PCA (p=0) + fine-tuning, PUIS pour chaque p du
#   balayage, remplacement apres coup d'une fraction p des filtres
#   deja entraines par du pseudo-aleatoire, SANS re-entrainer a
#   nouveau (feuille Avec_reentrainement_PCA). Utilise
    epochs_retrain_pca.
# retrain_after_pseudo : compresse chaque (eth, p) du balayage puis
#   fine-tune directement avec le p reel (feuille
    Avec_reentrainement_Pseudo).
#   Utilise epochs_retrain_pseudo.
retrain_after_pca    = false   # true / false  (oui / non, 1 / 0)
retrain_after_pseudo = false
epochs_initial        = 4
epochs_retrain_pca    = 2   # epoques de FT pour retrain_after_pca
epochs_retrain_pseudo = 2   # epoques de FT pour retrain_after_pseudo
lr         = 0.001
batch_size = 64
```

```
freeze_fc  = false   # geler les couches Linear pendant le
    ré-entraînement

# -- Pseudo-randomisation ------------------------------------------
seed = 42    # graine LFSR
K    = 100   # candidats par vecteur LFSR

# -- Sorties -------------------------------------------------------
save_models = false
save_dir    = checkpoints
output      =                # vide ->
    resultats_MODELE_DATASET_AAAAMMJJ_HHMMSS.xlsx
output_dir  = results
log_dir     = logs
cache_dir   = cache
no_cache    = false

# -- Divers --------------------------------------------------------
# device = cuda           # decommenter pour forcer le GPU ou le CPU
# baseline_only = true    # n'entrainer et n'evaluer que le modele
    initial
```

Listing 7: Contenu de `config_exemple.txt`.

3.1.4 Test de compression sur plusieurs modèles

Pour vérifier rapidement qu'une modification du pipeline (nouvelle option, refactorisation de `nrc/compress.py`, etc.) n'a pas cassé la compression sur une architecture donnée, le script `test_all_models.py` fourni avec le dépôt lance automatiquement `run.py` sur chaque modèle enregistré dans `MODEL_REGISTRY`, avec un budget volontairement réduit (1 époque, un seul *eTh*, deux valeurs de p dont $p = 0$, peu de candidats LFSR) : l'objectif n'est pas de mesurer l'accuracy ou le gain de compression, mais de détecter en quelques minutes une régression qui ferait planter le pipeline (erreur de forme, couche groupée non supportée — cf. §3.3 — etc.).

```
python test_all_models.py                          # tous les modèles
python test_all_models.py --models lenet resnet18   # sous-ensemble
python test_all_models.py --dataset cifar100        # avec CIFAR-100
python test_all_models.py --full                    # + retrain
    PCA/pseudo
```

Listing 8: Test de compression sur plusieurs modèles (`test_all_models.py`).

Chaque modèle est testé dans son **propre sous-processus** Python (appel de `run.py` via `subprocess`), ce qui évite toute pollution d'état (mémoire CUDA, graines) entre

deux modèles testés à la suite. Un résumé `OK`/`ÉCHEC` par modèle est affiché en fin d'exécution, avec la fin de la sortie d'erreur pour chaque échec ; le script termine avec un code de sortie non nul si au moins un modèle échoue, ce qui permet de l'intégrer tel quel dans un script d'intégration continue. L'option `--full` active en plus `--retrain-after-pca` et `--retrain-after-pseudo` (1 époque de fine-tuning chacun), pour exercer également le pipeline séquentiel PCA+FT décrit au §9 et pas seulement la compression brute.

3.2 Lecture des résultats

3.2.1 Gain de compression

Le **gain de compression** est défini comme le rapport entre le nombre de paramètres convolutifs du modèle de référence et celui du modèle NRC :

$$G_{\text{compression}} = \frac{N_{\text{params,ref}}}{N_{\text{params,NRC}}}. \tag{1}$$

Un gain supérieur à 1 indique que la compression NRC permet de réduire le nombre de paramètres convolutifs.
Le gain augmente généralement lorsque *eTh* diminue, car moins de vecteurs ou filtres de convolutions issues de PCA sont conservées.
La proportion *p* de pseudo-randomisation ne réduit pas directement le nombre de dimensions de la factorisation. Elle agit principalement sur le stockage puisque les vecteurs pseudo-aléatoires sont remplacés par leurs graines de 4 octets.

3.2.2 Écart d'accuracy

La différence d'accuracy par rapport à la baseline est donnée par :

$$\Delta Acc = Acc_{\text{NRC}} - Acc_{\text{baseline}}. \tag{2}$$

Cet écart représente le coût de la compression. Le fine-tuning partiel du bloc 5 vise à récupérer cette perte d'accuracy.

3.2.3 Coût de calcul (MAC)

Le gain de compression (nombre de paramètres) ne dit rien du coût de calcul réel d'une inférence : c'est le rôle du **MAC** (*Multiply-ACcumulate*), l'unité de mesure standard du coût d'une convolution, calculé par `count_conv_macs` (`nrc/compress.py`) via des hooks forward sur le modèle.

Couche d'origine. Pour une `Conv2d` classique :

$$\mathrm{MAC}_{\mathrm{ref}} = c_{out} \times d \times H_{out} \times W_{out}, \tag{3}$$

avec $d = c_{in}/\mathrm{groups} \times k_H \times k_W$ (même convention qu'au bloc 2) et H_{out}, W_{out} les dimensions spatiales de sortie.

Couche compressée. Pour un `NRCConvBlock` :

$$\mathrm{MAC}_{\mathrm{NRC}} = \left[\mathrm{groups} \times d \times (t+1) + t \times c_{out}\right] \times H_{out} \times W_{out}, \qquad t = n_{\mathrm{keep}} + n_{\mathrm{rand}} = s. \tag{4}$$

Le terme $\mathrm{groups} \times d \times (t+1)$ couvre `conv_keep` et les vecteurs pseudo-aléatoires régénérés (`rand_weight`, dont le calcul a un coût réel même s'il n'est pas stocké) ainsi que `mean_conv` ; le terme $t \times c_{out}$ couvre `conv_coord`, la couche de recombinaison.

Gain MAC. Par analogie avec le gain de compression :

$$G_{\mathrm{MAC}} = \frac{\mathrm{MAC}_{\mathrm{ref}}}{\mathrm{MAC}_{\mathrm{NRC}}}. \tag{5}$$

Contrairement au gain de compression, G_{MAC} peut être **inférieur à 1** : la décomposition en base + recombinaison peut coûter plus d'opérations qu'une convolution directe, même lorsqu'elle réduit la mémoire occupée. Les colonnes `MAC` et `Gain MAC` du fichier Excel (bloc 6) permettent de repérer ces cas configuration par configuration, plutôt que de supposer que gain de compression et gain de calcul évoluent toujours dans le même sens.

3.3 Ajout d'un nouveau modèle

Le pipeline est indépendant de l'architecture dans une certaine mesure : `nrc/compress.py` détecte automatiquement toutes les couches `Conv2d` d'un `nn.Module` via `model.named_modules()`.

Architectures compatibles. En l'état, la compression NRC convient à toute architecture bâtie exclusivement sur des convolutions standards : **LeNet**, **VGG16** et **ResNet18/50** (dans leurs variantes usuelles `models/resnet18.py` et `models/resnet50.py`, qui n'utilisent aucune convolution groupée) fonctionnent nativement — ce sont les architectures déjà validées dans ce dépôt.

Architectures incompatibles en l'état. À l'inverse, toute architecture reposant sur des convolutions *depthwise separable* ou groupées échoue avec le code actuel : c'est le cas de **MobileNetV1/V2/V3**, **EfficientNet**, **ShuffleNet**, ou de toute variante *ResNeXt*. Pour **MobileNetV2** en particulier, chaque bloc *inverted residual* alterne des convolutions 1×1 standards (compatibles) et une convolution 3×3 *depthwise* avec `groups=c_in` (incompatible) : la compression échoue dès la première couche depthwise rencontrée.
Ajouter le support des couches groupées nécessiterait de propager `groups` depuis la `Conv2d` d'origine jusqu'à `NRCConvBlock` (constructions de `conv_keep`, `conv_coord` et `mean_conv` avec `groups=...`), et d'adapter `FilterAnalyzer` / `PseudoRandomFilterSelector` pour analyser chaque groupe indépendamment — ce qui reste à faire.
Ajouter un nouveau modèle *compatible* (n'utilisant que des convolutions standards) ne nécessite donc de modifier que deux endroits :

1. créer un nouveau fichier dans `models/`, définissant une classe `nn.Module` dont la sortie est compatible CIFAR-10 (entrée (3, 32, 32), sortie à `num_classes` logits, par défaut 10) ;
2. enregistrer un point d'entrée `nom` → `callable(args)` dans le dictionnaire `MODEL_REGISTRY` de `run.py`.

```python
# models/mon_modele.py
import torch
import torch.nn as nn

class MonModele(nn.Module):
    def __init__(self, num_classes: int = 10):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, 3, padding=1), nn.ReLU(inplace=True),
            nn.MaxPool2d(2),                        # -> 16x16
        )
        self.classifier = nn.Linear(32 * 16 * 16, num_classes)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.features(x)
        x = torch.flatten(x, 1)
        return self.classifier(x)
```

Listing 9: Squelette minimal d'un nouveau modèle CIFAR-10.

```python
# run.py -- dans MODEL_REGISTRY
from models.mon_modele import MonModele

MODEL_REGISTRY = {
    ...
    "mon_modele": lambda args: MonModele(),
}
```

Listing 10: Enregistrement du modèle dans `MODEL_REGISTRY`.

Le modèle devient alors utilisable via `–model mon_modele` sans aucune autre modification du pipeline : l'analyse PCA, la pseudo-randomisation LFSR, le `NRCConvBlock`, le fine-tuning et l'export Excel s'appliquent automatiquement à ses couches `Conv2d`. Deux points ne sont pas détectés automatiquement à l'exécution et doivent être vérifiés manuellement lors de l'ajout d'un modèle :

- **Résolution spatiale** : une architecture conçue pour ImageNet (par exemple `torchvision.models.mobilenet_v2`, pensée pour des images 224×224) réduit généralement la résolution d'un facteur 32. Appliquée telle quelle à CIFAR-10 (32×32), la carte de caractéristiques finale peut s'effondrer à 1×1 avant le pooling global, ce qui dégrade fortement l'accuracy sans provoquer d'erreur. Il est recommandé d'adapter le stride des premières couches, comme dans `models/resnet18.py` (`conv1` en stride 1 au lieu de stride 2, pas de `MaxPool2d` après `conv1`).

- **Couches non convolutives** : seules les `Conv2d` sont compressées par la méthode NRC ; les couches `nn.Linear` restent inchangées (éventuellement gelées via `–freeze-fc`). Un modèle dont la majorité des paramètres est portée par des couches `Linear` (têtes de classification volumineuses, architectures à base d'attention) ne bénéficiera donc que marginalement de la compression.

4 Vue d'ensemble du flot de conception

La méthode NRC est organisée en six blocs successifs. Chaque bloc réalise une fonction spécifique et produit les informations nécessaires au bloc suivant.
Le flot global peut être résumé comme suit :

1. entraînement initial du modèle ;
2. analyse PCA des filtres convolutifs ;

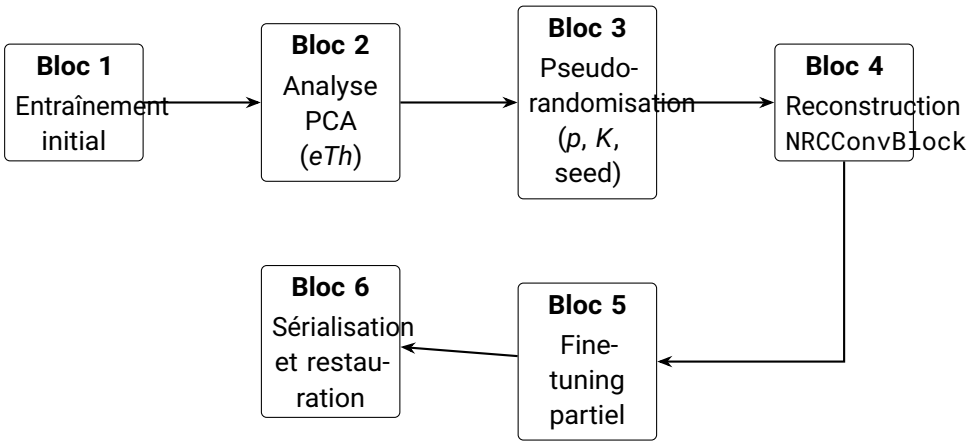


Figure 1: Flot global de conception de la méthode NRC.

3. pseudo-randomisation d'une partie de la base PCA ;
4. reconstruction des couches sous forme de `NRCConvBlock` ;
5. fine-tuning partiel du modèle compressé ;
6. sérialisation, restauration et évaluation finale.

4.1 Validation par des balayages des paramètres

En cas de performances insuffisantes, le flot est réorienté vers le bloc responsable, selon les critères résumés dans le tableau 3 et les actions correctives à faire manuellement.

5 Bloc 1 — Entraînement du modèle initial

5.1 Objectif

Le premier bloc consiste à entraîner le modèle de référence avant toute opération de compression.
L'objectif est d'obtenir les poids convolutifs de référence W et de définir les performances initiales du réseau. Ces résultats servent ensuite de référence pour évaluer l'impact de la compression, que ce soit en termes d'accuracy, de nombre de paramètres ou de mémoire occupée.

Critère de non-satisfaction	Action corrective
Accuracy de référence insuffisante	Ajuster le nombre d'époques ou le learning rate (bloc 1).
Accuracy compressée ou gain de compression insuffisants	Modifier *eTh* ou *p* (blocs 2 et 3).
Écart Δ trop important vis-à-vis de la baseline	Ajuster le fine-tuning partiel (bloc 5).
Accuracy après fine-tuning différente de l'accuracy finale	Vérifier les graines et la fiche technique (manifeste.json) (bloc 6, `assert` dans `pipeline.py`).

Table 3: Critères de validation et rebouclage du flot.

5.2 Fichiers concernés

- `run.py`
- `models/lenet.py`
- `models/vgg16.py`
- `models/vgg_mini.py`
- `models/resnet18.py`
- `models/resnet50.py`

5.3 Entrées et sorties

Élément	Description
Entrées	Jeu de données sélectionné dans `DATASET_REGISTRY` via `–dataset` (CIFAR-10 ou CIFAR-100, torchvision, normalisation $(0.5, 0.5, 0.5)$) et architecture sélectionnée dans `MODEL_REGISTRY`.
Sorties	Modèle entraîné contenant les poids W de chaque couche `Conv2d` et accuracy de référence.
Métriques	Accuracy de référence (validation), nombre de paramètres convolutifs et mémoire occupée par les paramètres convolutifs.

Table 4: Entrées, sorties et métriques du bloc 1.

5.4 Paramètres de configuration

Option	Défaut	Rôle
`–model`	`lenet`	Choix de l'architecture : `lenet`, `vgg16`, `vgg_mini`, `resnet18`, `resnet50`.
`–dataset`	`cifar10`	Choix du jeu de données : `cifar10` ou `cifar100` (§2.2). Détermine `num_classes`, propagé à `MODEL_REGISTRY`.
`–epochs-initial`	4	Nombre d'époques d'entraînement initial.
`–lr`	10^{-3}	Learning rate utilisé par Adam.
`–batch-size`	64	Taille du batch.
`–n-filters`	6	Nombre de filtres de la première couche convolutive (LeNet uniquement).
`–baseline-only`	off	Permet de réaliser uniquement l'entraînement et l'évaluation de référence.
`–device`	auto	Utilisation de CUDA si disponible, sinon du CPU.

Table 5: Paramètres du bloc d'entraînement initial.

6 Bloc 2 — Analyse PCA des filtres

6.1 Objectif

Le deuxième bloc vise à réduire la dimension (n_f : *nombre de filtres de convolutions*) de représentation de chaque couche convolutive, en la projetant sur une base compacte de s vecteurs propres, avec $s \ll n_f$.

6.2 Fichier concerné

- `nrc/analyze.py` (classe `FilterAnalyzer`)

6.3 Centrage et réduction de dimension

La matrice des filtres entraînés W est définie par :

$$W \in \mathbb{R}^{d \times n_f}, \tag{6}$$

où

- $d = c_{\text{in}} \times k_H \times k_W$
  tel que:
  - c_{in}: nombre de canaux d'entrée de la couche convolutive (par exemple: 3 pour une image RGB en entrée de réseau);
  - k_H : hauteur du noyau de convolution
  - k_W : largeur du noyau de convolution
- n_f représente le nombre de filtres et où chaque colonne correspond à un filtre aplati.

Les filtres sont ensuite centrés par rapport au filtre moyen ($\overline{w}$) avant d'effectuer une analyse en composantes principales.

$$W_C = W - \overline{w}. \tag{7}$$

Le nombre s de composantes conservées est déterminé à partir du seuil d'énergie eTh :

$$s = \min \left\{ k : \frac{\text{trace}(D_k)}{\text{trace}(D)} \geq eTh \right\}. \tag{8}$$

où :

- D : matrice diagonale des valeurs propres de la **matrice de covariance** des filtres centrés W_C, triées par ordre décroissant (une valeur propre par composante principale, trace(D) représentant l'energie totale);
- D_k : matrice diagonale restreinte aux k premières (les plus grandes) valeurs propres de D, de sorte que trace(D_k) représente l'énergie cumulée capturée par les k premières composantes principales.

Puis, le nombre de composantes est borné par :

$$2 \leq s \leq \min(n_f, d). \tag{9}$$

afin de constituer une base vectorielle formé au moins par deux (02) vecteurs ou deux(02) filtres de convolutions.
Et enfin, la base issue de PCA retenue est ensuite :

$$B_T = s \text{ premiers vecteurs propres}, \qquad B_T \in \left(\mathbb{R}^d\right)^s. \tag{10}$$

Et les coordonnées des filtres projetées dans cette base sont calculées par :

$$Y_T = W_C^T B_T^T. \tag{11}$$

6.4 Entrées, sorties et métriques

Élément	Description
Entrées	Dictionnaire `{nom : Conv2d}` de l'ensemble des couches convolutives du modèle.
Sorties	B_T de dimension (s), Y_T, moyenne $\overline{w}$, indices importants et négligés, variance expliquée.
Métriques	Nombre de composantes conservées par couche (identique à la dimension s de la nouvelle base B_T), énergie capturée minimale donnée par l'équation (8) (en %).

Table 6: Entrées, sorties et métriques du bloc PCA.

6.5 Paramètre principal

Le paramètre principal est le seuil d'énergie *eTh* :

$$eTh \in [0, 1]. \tag{12}$$

La configuration par défaut utilise :

$$eTh = 0.90. \tag{13}$$

Plus *eTh* est faible, moins de composantes sont conservées, ce qui augmente le potentiel de compression.

Option	Défaut	Rôle
`–eth`	0.90	Seuil d'énergie. Plusieurs valeurs peuvent être fournies afin de réaliser un balayage, par exemple `–eth 0.25 0.50 0.75 0.90`.

Table 7: Paramètre du bloc d'analyse PCA.

7 Bloc 3 — Pseudo-randomisation LFSR et distance de Grassmann

7.1 Objectif

Le troisième bloc consiste à remplacer une proportion p des vecteurs PCA par des vecteurs pseudo-aléatoires recalculable à partir de leur graine de 4 octets, générés par un LFSR, au lieu de stocker toutes les filtres de dimension d.

7.2 Fichiers concernés

- `nrc/select.py` (classe `PseudoRandomFilterSelector`)
- `utils.py` (`_prng_vector`, `_grassmann_distance`)

7.3 Répartition des vecteurs

Pour un nombre total de composantes s, le nombre de vecteurs PCA conservés est :

$$n_{\text{keep}} = \max\left(1, \text{round}\left(s(1-p)\right)\right), \tag{14}$$

où *round* applique un **arrondi au pair le plus proche**.
Et, les n_{keep} vecteurs ainsi conservés constituent précisément la base V_S
Puis, le nombre de vecteurs pseudo-aléatoires à regénerer est donné comme suit:

$$n_{\text{rand}} = s - n_{\text{keep}}. \tag{15}$$

Ces n_{rand} vecteurs constituent la base V_r
D'où, la base hybride finale est la concaténation de ces deux bases:

$$B_{\text{NRC}} = [V_s \mid V_r] \tag{16}$$

7.4 Sélection des vecteurs pseudo-aléatoires

La sélection est réalisée de manière gloutonne. Pour chaque vecteur PCA devant être remplacé :

1. K graines candidates sont tirées dans $[0, 2^{31})$;
2. chaque graine permet de générer un vecteur LFSR de dimension d ;
3. le vecteur est normalisé, divisé par sa norme euclidienne pour obtenir un vecteur de norme unitaire ;

4. la distance de Grassmann est calculée entre deux sous-espaces vectoriels : *(i)* le sous-espace cible, engendré par les s vecteurs propres complets de B_T (la base PCA d'origine, avant tout remplacement) et *(ii)* le sous-espace en cours de construction, formé des vecteurs déja retenus (les n_{keep} vecteurs PCA conservés, ainsi que les vecteurs pseudo-aléatoires déja sélectionnés aux itérations précédentes de génération) auquel on ajoute provisoirement le candidat testé. Plus cette distance est faible, plus le sous-espace hybride reste proche du sous-espace PCA original, c'est ce critère qui guide le choix de la graine à chaque itération.

5. puis, on sélectionne la graine donnant la distance minimale

Il existe plusieurs types Le LFSR, et on a utilisé un LFSR Fibonacci 32 bits avec les taps ou les positions qui vont être combinés par une chaîne de portes XOR, commme illustré dans la figure 2, sont les suivantes:

$$[32, 22, 2, 1]. \tag{17}$$

correspondant à ce polynôme de rétroaction :

$$x^{32} + x^{22} + x^2 + 1 \quad \Longleftrightarrow \quad \texttt{0x80200003}. \tag{18}$$

La figure 2 illustre ce mécanisme : à chaque étape, les bits situés aux positions des taps sont combinés par une chaîne de portes XOR pour former le bit de feedback, le registre est décalé d'une position (le bit 1, en sortie, est perdu) et le feedback obtenu est injecté à l'entrée libérée, au poids fort (position 32).

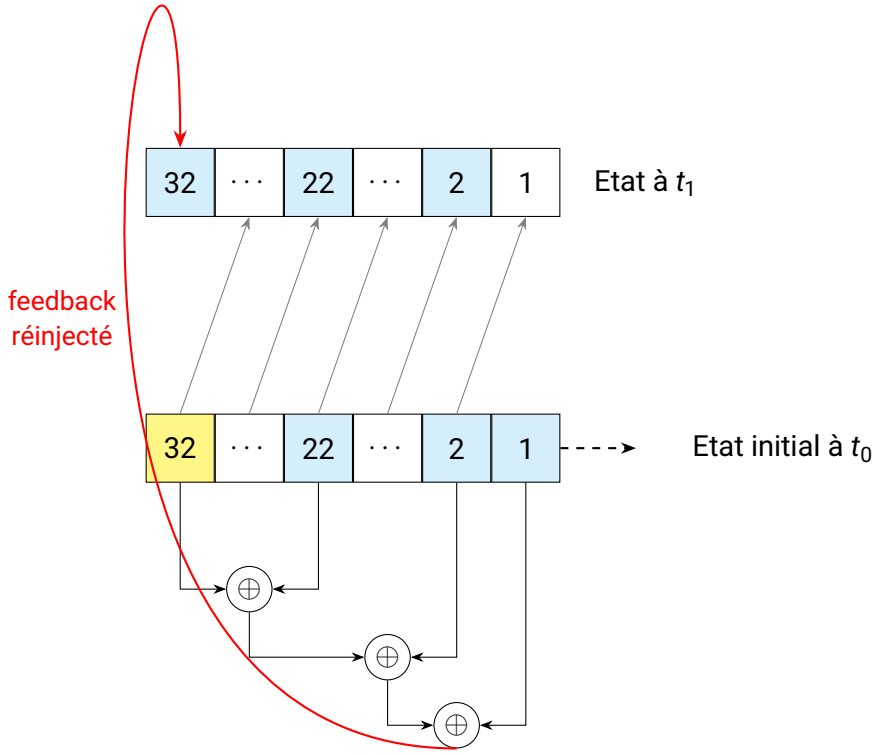


Figure 2: Principe d'un LFSR Fibonacci sur 32 bits (taps utilisés dans le pipeline : positions 32, 22, 2 et 1, en bleu clair ; les positions intermédiaires, non tapées, sont symbolisées par « · · · »). Passage de l'état t_0 à l'état t_1 : *(i)* les bits aux positions des taps sont combinés par une chaîne de XOR pour produire le bit de feedback ; *(ii)* tout le registre est décalé d'une position vers la droite (flèches grises), le bit 1 sortant étant perdu ; *(iii)* le feedback est injecté à l'entrée libérée, en jaune (poids fort, position 32).

La transition $t_0 \to t_1$ illustrée ci-dessus ne produit qu'un seul bit (une seule composante scalaire). Pour construire un vecteur pseudo-aléatoire complet de dimension $d = c_{\text{in}} \times k_H \times k_W$, cette transition est répétée d **fois** (une par composante du vecteur) : ce nombre n'a pas de lien direct avec K, qui désigne le nombre de *vecteurs candidats distincts* testés (chacun généré à partir d'une graine différente, donc nécessitant lui aussi d répétitions) avant de retenir celui qui minimise la distance de Grassmann. La sélection d'un seul vecteur pseudo-aléatoire active donc le LFSR $K \times d$ fois au total. Puis, les valeurs générées sont ramenées dans l'intervalle $(-0.5, 0.5)$ puis normalisées au sens de la norme L_2.

7.5 Distance de Grassmann

La distance de Grassmann permet d'évaluer la proximité entre le sous-espace vectoriel PCA cible et le sous-espace vectoriel obtenu avec la base hybride. Il s'agit de la distance géodésique entre sous-espaces, qui se calcule non pas directement entre les vecteurs des deux bases, mais à partir des **angles principaux** qui séparent les deux sous-espaces qu'elles engendrent : plus ces angles sont petits, plus les sous-espaces sont proches, et plus la distance géodésique qui en résulte est faible. Ces angles principaux sont obtenus à partir d'une décomposition SVD, comme détaillé ci-dessous ; lorsqu'une dimension n'a pas de correspondance entre les deux bases, une pénalité de $\pi/2$ est appliquée (cf. paragraphe suivant).
Une forme générale de la distance utilisée est :

$$d_G = \left(\sum_{i=1}^{j} \theta_i^2 + (t - j) \left(\frac{\pi}{2} \right)^2 \right)^{1/2} . \tag{19}$$

Une faible distance indique que la base pseudo-randomisée conserve une bonne proximité avec le sous-espace PCA cible.

7.6 Entrées, sorties et métriques

Élément	Description
Entrées	Résultats de PCA du bloc 2, soit B_T pour chaque couche.
Sorties	Base hybride $B_{\text{NRC}} = [V_s \mid V_r]$, liste des graines sélectionnées (`selected_seeds`, entiers 32 bits) et historique des distances pour chaque vecteur remplacé par un vecteur pseudo-aléatoire.
Métriques	Distance de Grassmann finale vis-à-vis de B_T, répartition $n_{\text{keep}}/n_{\text{rand}}$ par couche.

Table 8: Entrées, sorties et métriques du bloc de pseudo-randomisation.

7.7 Paramètres de configuration

Option	Défaut	Rôle
–p	0.50	Proportion de randomisation : p = 0 correspond à une base entièrement PCA, p = 1 à une base entièrement LFSR. Plusieurs valeurs peuvent être fournies.
–K	100	Nombre de graines candidates testées pour chaque vecteur remplacé.
–seed	42	Graine de base, incrémentée de +1 à chaque couche de convolution.

Table 9: Paramètres du bloc de pseudo-randomisation.

8 Bloc 4 — Reconstruction de couches de convolutions (ou NRCConvBlock)

8.1 Objectif

Le quatrième bloc remplace chaque couche `Conv2d` par un bloc `NRCConvBlock` équivalent, tout en conservant les propriétés architecturales telles que le stride, le padding, la dilatation et le biais. Le remplacement est réalisé sans modification du reste de l'architecture.

8.2 Fichiers concernés

- `nrc/block.py` (classe `NRCConvBlock`)
- `nrc/compress.py` (fonction `compress`)

8.3 Formulation

La reconstruction est donnée par :

$$y = \mathrm{conv}_{coord}\left(\mathrm{conv}_{base}(x)\right) + \mathrm{mean}_{conv}(x). \tag{20}$$

Les coordonnées dans la base hybride sont calculées par :

$$Y_{\text{NRC}} = W_C^T B_{\text{NRC}}^T. \tag{21}$$

8.4 Composition du bloc NRCConvBlock

Composante	Poids	Dimensions	Statut
`conv_base`	B_{NRC}	(s, d) vu comme (s, c_{in}, k_H, k_W)	Gelé
`conv_coord`	Y_{NRC} + biais	$(c_{out}, s, 1, 1)$	Entraînable
`mean_conv`	$\overline{w}$	$(1, c_{in}, k_H, k_W)$	Gelé

Table 10: Composition du NRCConvBlock.

Le principal avantage de cette organisation est de conserver explicitement uniquement une partie de la base, tandis que les vecteurs pseudo-aléatoires peuvent être régénérés à partir de leurs graines.

8.5 Entrées, sorties et métriques

Élément	Description
Entrées	Modèle source (copié par `deepcopy`), B_{NRC}, Y_{NRC}, $\overline{w}$ et biais d'origine.
Sorties	Modèle NRC obtenu par remplacement direct des couches, ainsi que le fichier technique `manifest.json` contenant *eTh*, *p*, *K*, la graine de base pour la reconstruction du modèle compressé. et, pour chaque couche, la forme d'origine, s, n_{keep}, n_{rand}, les graines, `BNRC_keep` et $\overline{w}$.
Métriques	Nombre de paramètres convolutifs avant et après compression, par couche, et gain de compression.

Table 11: Entrées, sorties et métriques du bloc de reconstruction.

8.6 Paramètres de configuration

Option	Défaut	Rôle
`freeze_linear` (`–freeze-fc`)	off	Gèle l'ensemble des couches `nn.Linear`.

Table 12: Paramètre du bloc de reconstruction.

Le bloc est générique : il s'applique à toute architecture contenant des couches `Conv2d`.

9 Bloc 5 — Fine-tuning partiel

9.1 Objectif

La compression provoque généralement une perte d'accuracy. Le cinquième bloc permet donc de récupérer une partie de cette perte grâce à un fine-tuning partiel car on re-ajuste uniquement les coordonnées (Y_T ou Y_{NRC}), mais tous les poids des filtres ou des vecteurs dans la base réduite (B_T ou B_{NRC}), le vecteur moyen $\overline{w}$, et les poids des couches FC qui vont être gelées ou fixées.

9.2 Fichiers concernés

- `nrc/train.py` (`train_one_epoch`, `evaluate`)
- `run.py` (fonction `_finetune`)

9.3 Équité de comparaison

Afin de comparer équitablement le modèle de référence et le modèle compressé, le budget d'entraînement total est conservé pour chaque branche de ré-entraînement :

$$e^b_{\text{pr}-\text{NRC}} + e^b_{\text{retrain}} = e_{\text{initial}} \quad (\text{pour } b \in \{\text{pca}, \text{pseudo}\}) \tag{22}$$

Ainsi, le modèle est entraîné avant compression pendant :

$$e^b_{\text{pr}-\text{NRC}} = e_{\text{initial}} - e^b_{\text{retrain}} \quad (\text{nombre d'époques pour le ré-entrainement partiel}) \tag{23}$$

–epochs-retrain-pca et –epochs-retrain-pseudo étant indépendants, un seul modèle pré-NRC partagé ne peut pas satisfaire les deux équations simultanément dès qu'ils diffèrent : run.py entraîne donc **deux** modèles pré-NRC distincts (net_pretrain_pca, net_pretrain_pseudo — fonction _pretrain_for), chacun uniquement si le flag –retrain-after-pca / –retrain-after-pseudo correspondant est actif.

9.4 Entrées, sorties et métriques

Élément	Description
Entrées	Modèle NRC et DataLoader d'entraînement CIFAR-10.
Sorties	Modèle NRC ré-entraîné et courbes de loss et d'accuracy par époque.
Métriques	Accuracy avant et après fine-tuning, écart Δ vis-à-vis de la baseline, loss d'entraînement et loss de validation.

Table 13: Entrées, sorties et métriques du fine-tuning partiel.

9.5 Paramètres de configuration

Option	Défaut	Rôle
`–retrain-after-pca`	off	Fine-tuning après l'étape PCA ($p = 0$, ou Bloc 2), PUIS pour chaque p du balayage, remplacement après coup d'une fraction p des filtres déjà entraînés par du pseudo-aléatoire SANS ré-entraîner à nouveau (feuille Excel `Avec_reentrainement_PCA`).
`–retrain-after-pseudo`	off	Compresse chaque (eTh, p) du balayage puis fine-tune directement avec le p réel (ou Bloc 3+4), feuille `Avec_reentrainement_Pseudo`.
`–epochs-retrain-pca`	2	Nombre d'époques de ré-entraînement pour la branche `–retrain-after-pca`.
`–epochs-retrain-pseudo`	2	Nombre d'époques de ré-entraînement pour la branche `–retrain-after-pseudo`.
`–freeze-fc`	off	Gèle les couches FC pendant le fine-tuning.

Table 14: Paramètres du bloc de fine-tuning partiel.

10 Bloc 6 — Sérialisation, restauration et bilan

10.1 Objectif

Le dernier bloc permet de sauvegarder le modèle compressé sans stocker explicitement les vecteurs pseudo-aléatoires.
Chaque vecteur pseudo-aléatoire est régénéré lors du chargement à partir de sa graine LFSR de 4 octets.
Le processus de restauration doit permettre de retrouver exactement les performances du modèle final. Le contrôle est réalisé selon :

$$|Accuracy_{\text{restaure}} - Accuracy_{\text{finale}}| < 10^{-4}. \tag{24}$$

Cette équation ne porte que sur *un seul* modèle restauré : elle garantit la fidélité de la sérialisation, pas encore le compromis accuracy / mémoire dans son ensemble. Ce dernier est quantifié par un mécanisme différent : pour **chaque** configuration (eTh, p) du balayage, `run.py` enregistre dans une ligne de tableau l'accuracy obtenue, le nombre de paramètres convolutifs, la mémoire correspondante et le gain de compression associé (cf. bloc 1, équation de M_{conv}, et définition du gain de compression donnée en section « Lecture des résultats »). Une fois **toutes** les configurations de la grille $eTh \times p$ traitées, ces lignes sont assemblées dans le fichier Excel décrit ci-après (tableau 15). C'est cette table complète — et non l'équation de restauration — qui « quantifie le compromis » : elle permet de lire, pour chaque point de la grille, l'accuracy obtenue en fonction du gain de compression, et donc de comparer objectivement les configurations entre elles plutôt que de n'en évaluer qu'une seule isolément.

10.2 Fichiers concernés

- `nrc/serialize.py` (`save_manifest`, `load_manifest`, `save_compressed_model`)
- `run.py` (export Excel)
- `pipeline.py` (vérification de la restauration)

10.3 Fichiers produits

Fichier	Contenu
`<prefix>_state.pth`	`state_dict` PyTorch contenant les poids entraînables.
`<prefix>_manifest.json`	Métadonnées et graines utilisées, dans un format lisible.
`<prefix>_manifest_arrays.npz`	Tableaux NumPy contenant `BNRC_keep` et $\overline{w}$ pour chaque couche.
`resultats_<horodatage>.xlsx`	Résultats expérimentaux selon les différentes configurations $eTh \times p$, répartis en trois feuilles : `Sans_reentrainement`, `Avec_reentrainement_PCA` et `Avec_reentrainement_Pseudo`.

Table 15: Fichiers générés lors de la sérialisation.

Couleur	Signification
Vert	Accuracy supérieure ou égale à la référence ; paramètres ou mémoire meilleurs que la référence.
Jaune	Accuracy comprise entre la référence et la référence diminuée de 5 points.
Rouge	Accuracy inférieure à ce seuil ; paramètres ou mémoire moins bons que la référence.
Bleu	Colonnes relatives aux graines.

Table 16: Code couleur utilisé dans le bilan Excel.

10.4 Code couleur du fichier Excel

10.5 Entrées, sorties et métriques

Élément	Description
Entrées	Modèle NRC affiné, manifeste associé et jeu de test CIFAR-10.
Sorties	Fichiers décrits dans le tableau 15 et modèle restauré.
Métriques	Nombre de graines et volume correspondant (nb $\times$ 4 octets), mémoire convolutive finale, accuracy restaurée et gain de compression.

Table 17: Entrées, sorties et métriques du bloc de sérialisation.

10.6 Paramètres de configuration

Option	Défaut	Rôle
`–save-models`	off	Sauvegarde chaque modèle compressé accompagné de son manifeste.
`–save-dir`	`checkpoints`	Répertoire de sauvegarde des modèles.
`–output`	horodaté	Nom du fichier Excel produit.
`–output-dir`	`results`	Dossier des exports.
`–log-dir`	`logs`	Dossier des journaux console et fichiers horodatés.

Table 18: Paramètres du bloc de sérialisation.

10.7 Bilan expérimental

Le fichier Excel permet de comparer plusieurs configurations et d'étudier le compromis entre :

- **`accuracy`**: Taux de classification correcte (%) sur le jeu de test CIFAR-10, calculé par `evaluate` (`nrc/train.py`). Rapportée selon jusqu'à trois variantes par ligne : accuracy du modèle de référence, accuracy avant fine-tuning et accuracy après fine-tuning (colonnes distinctes selon la feuille, cf. tableau 15).
- **`nombre de paramètres`** : Nombre de paramètres portés uniquement par les couches Conv2d/NRCConvBlock (`count_conv_params`, `nrc/compress.py`);
- **`mémoire`** : Mémoire occupée par ces mêmes paramètres en précision `float32`, en Ko : $M_{\text{conv}} = N_{\text{params}} \times 4/1024$ (formule du bloc 1, colonne « Mémoire conv (KB) »).;
- **`gain de compression`** : Rapport $G_{\text{compression}} = N_{\text{params,ref}}/N_{\text{params,NRC}}$ entre le nombre de paramètres conv du modèle de référence et celui du modèle compressé à ce point de la grille (défini en section « Lecture des résultats »). Un gain > 1 signifie une compression effective. ;
- **`nombre de graines utilisées`** : Somme, sur **toutes** les couches compressées du modèle, du nombre de graines LFSR sélectionnées

(`selected_seeds`) — colonne « Seeds (nb) », calculée par `_count_seeds` (`run.py`). Comme cette somme porte sur l'ensemble du réseau (et non sur une seule couche), elle dépasse généralement *K* (cf. discussion du bloc 3). La colonne « Seeds (octets) » correspondante vaut ce nombre $\times$ 4 : c'est le coût de stockage réel de la partie pseudo-aléatoire, puisque seules les graines (4 octets chacune) sont conservées, jamais les vecteurs eux-mêmes. ;

11 Conclusion

Le flot de conception NRC permet de transformer progressivement un réseau convolutif classique en une représentation compressée basée sur une combinaison de réduction dimensionnelle et de pseudo-randomisation.
Le processus complet repose sur six étapes principales :

1. entraînement du modèle de référence ;
2. analyse PCA des filtres ;
3. sélection des vecteurs pseudo-aléatoires par LFSR et distance de Grassmann ;
4. reconstruction des couches sous forme de `NRCConvBlock` ;
5. fine-tuning partiel ;
6. sérialisation et vérification de la restauration.

L'évaluation finale permet alors d'étudier le compromis entre **accuracy**, **gain en compression mémoire** et **coût de calcul MAC** pour différentes valeurs de *eTh* et *p*.